# Conflicting Node Discrimination Graph Neural Network for Semi-supervised Node Classification

Wenjun Wang
University of Electronic Science and Technology of China
Chengdu, China
202522080926@std.uestc.edu.cn

Xin Cao
University of Electronic Science and Technology of China
Chengdu, China
x.cao@uestc.edu.cn

Yawen Li
Beijing University of Posts and Telecommunications
Beijing, China
warmly0716@bupt.edu.cn

XiaoLong Deng*
Beijing University of Posts and Telecommunications
Beijing, China
shannondeng@bupt.edu.cn

Kai Zheng*
University of Electronic Science and Technology of China
Chengdu, China
zhengkai@uestc.edu.cn

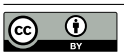
## Abstract

Graph Neural Networks (GNNs) have exhibited extraordinary performance in data mining. The success depends on the message passing mechanism, which is consistent with the homophily assumption that adjacent nodes share similar behaviors. However, the connected nodes in real-world graphs often show dissimilar behaviors, which is presented as the heterophily problem. A series of works proposed various models to solve the heterophily problem. They usually assume that the neighborhood distribution of nodes with the same class tends to be similar, while distinguishable from nodes of different classes. However, our experiments discovered that many nodes belonging to different classes share the same neighbor distribution in the mainstream datasets. The performance of GNN also degrades in these conflicting nodes. To detect the conflicting nodes and improve their prediction accuracy, we propose the Conflicting Node Discrimination GNN (CNDGNN). It firstly detects the possible conflicting nodes with the same neighbor distribution according to the selected high-quality pseudo-label. Then it estimates the class type of conflicting nodes with the same neighbor distribution through nodes' high-order neighbor distributions and original features, and the aggregation weights are learned to assign positive weights for nodes of the same class type and negative weights for nodes of different classes. The theoretical analysis proves that CNDGNN performs better than GCN in the conflicting nodes and explores the key design view when solving the conflicting node problem. The experiments demonstrate the effectiveness of our method in the mainstream datasets and verify that our method performs better in conflicting nodes.

## CCS Concepts

• **Computing methodologies** → **Artificial intelligence**; **Neural networks**.

*Corresponding authors: Kai Zheng and XiaoLong Deng.



This work is licensed under a Creative Commons Attribution 4.0 International License.
*KDD '26, Jeju Island, Republic of Korea*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
https://doi.org/10.1145/3770855.3817635

## Keywords

Graph Neural Networks; Heterophily; Node Classification

**ACM Reference Format:**
Wenjun Wang, Xin Cao, Yawen Li, XiaoLong Deng, and Kai Zheng. 2026. Conflicting Node Discrimination Graph Neural Network for Semi-supervised Node Classification. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770855.3817635

## 1 Introduction

Graphs serve as a fundamental structure for representing relational data and are widely used in many vital web applications, such as recommendation systems [6, 7, 38, 40], sponsored search [9, 14], and web page classification [8]. With the growing need to effectively process such data, Graph Neural Networks (GNNs) [36] have gained significant prominence. GNNs capture complex dependencies through iterative message passing, where each node refines its representation by aggregating information from its neighbors. Although GNNs and their variants have been widely applied across different domains, they are often built upon an implicit homophily assumption [4], meaning that connected nodes are expected to share similar properties. For instance, in social networks [18], people with similar interests are more likely to be friends. However, many real-world graphs exhibit heterophily, where links are more common between dissimilar nodes. An example is the World Wide Web, where web pages on different topics are frequently connected through hyperlinks, such as a news website linking to a government portal or a scientific article referencing a blog post. In such heterophilic settings, the effectiveness of traditional GNNs can be significantly limited, as their aggregation mechanisms are primarily designed to exploit homophilic structures.

A series of methods are proposed to solve the problem of heterophily in graphs. Some works focus on aggregating informative messages from high-order neighbors [15, 33–35]. They aggregate nodes from multi-hop neighbors [1] or directly search distant but informative nodes through defined geometric relationships [25] or the KNN algorithm [11, 12]. Others aim at conducting similarity-based or signed propagation to generate the discriminative node

embeddings. They learn the signed weights by comparing the similarity of topological and feature information [3, 10, 32, 37]. As later work [23] finds that certain types of "good" heterophily exist, under which GCNs can achieve strong performance, a series of works learning the heterophily pattern come out. Some works [5, 21] design multiple channels to learn the heterophily pattern additionally. Other works [17, 31, 42] learn the neighbor distribution to enhance node embeddings.

When learning graph's heterophily patterns, previous works usually assume that the neighborhood distribution of nodes with the same class tends to be similar, while distinguishable from nodes of different classes. The case is shown in (a),(b),(c), and (d) of Figure 1, where nodes of different labels have distinguishable neighbor distributions. However, our experiments discovered that many nodes belonging to different classes share the same neighbor distribution. In Figure 1, the nodes of (b), (c), and (e) belong to different classes while sharing the same neighbor distribution, and these nodes are represented as conflicting nodes in our paper. Our experiment finds that the phenomenon of conflicting nodes is universal in the mainstream homophilic and heterophilic graph datasets. The performance of GNN also degrades severely in these conflicting nodes, as the same neighbor distribution makes their embeddings similar and harder to distinguish after universal graph convolution. The degradation of GNN's performance in conflicting nodes is also consistent with the previous theorem [23], which is analyzed in section 4.3.

To increase the prediction accuracy of nodes with conflicting neighbor distribution, the objective is to improve the discrimination of conflicting nodes' embeddings. The idea is that, for conflicting nodes with the same neighbor distribution, we try to predict their class type based on some indicators and then aggregate conflicting nodes of the same predicted class type using positive weights and nodes of different predicted class types using negative weights. In this way, the embeddings of conflicting nodes with the same class get closer, and the embeddings of different classes are pushed away, thus enhancing the discrimination ability of conflicting node embeddings. The prediction of conflicting nodes' class types can utilize two indicators. 1. High-order neighbor distribution. Although these nodes are conflicted in the 1-hop neighbor distribution, they are more discriminative considering the high-order neighbor distribution. 2. Original node feature. Theorem 1 in section 4.3 illustrates that the original node features are more discriminative than the features after GNN for conflicting nodes.

Therefore, in this paper, we propose a new method named Conflicting Node Discrimination GNN (CNDGNN) to tackle the challenge of conflicting nodes. CNDGNN consists of three modules. The conflicting node detection module predicts each node's neighbor distribution according to the high-quality pseudo-label judged by the prediction confidence, and detects the possible conflicting nodes with the same neighbor distribution. Then, the conflicting embedding discrimination module estimates the class type of conflicting nodes with the same neighbor distribution through the nodes' high-order neighbor distribution and original feature, and a weight matrix is learned to assign positive weights for nodes of the same class type and negative weights for nodes of different types for later aggregation. Finally, the neighbor aggregation module aggregates ego feature, neighbor feature, and conflicting embedding through common tricks like ego-neighbor separation and layer concatenation. Moreover, we theoretically analyze our method and prove that it performs better than GCN in the conflicting nodes. The theorem analysis also explores the key design view when solving the conflicting node problem.

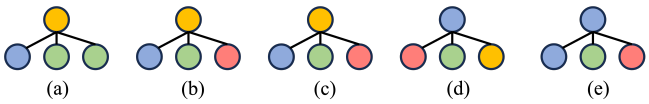



**Figure 1: Node's neighbor distribution. The figure shows a toy example of each node's neighbor distribution, where different colors represent different labels. The nodes in (b),(c), and (e) are represented as conflicting nodes.**

Our main contributions are as follows.

- We empirically observe that many nodes belonging to different classes share the same neighbor distribution in the mainstream homophilic and heterophilic datasets. We analyze the possible reasons and provide the theoretical analysis for its effect on the performance degradation based on an existing theorem.
- We propose a new method, CNDGNN, which detects the possible conflicting nodes and enhances the discrimination of the conflicting node embeddings to improve the prediction accuracy of semi-supervised node classification in both homophilic and heterophilic graphs. We theoretically prove that our method is effective in conflicting nodes and explore the key design view to address the challenge of dealing with conflicting nodes.
- The experiments demonstrate the effectiveness of our method and verify that our method performs better in conflicting nodes. The code is available at https://github.com/tourist-wwj/CNDGNN.

## 2 Related Work

To solve the problem of heterophily in graphs, a series of methods are proposed [20]. H2GCN [43] proposes three key designs: ego- and neighbor-representation separation, high-order neighborhood aggregation, and intermediate layers combination. These tricks are effective and widely applied in subsequent works. Some of the later works focus on aggregating messages from high-order neighbors [16, 24]. MixHop [1] adaptively aggregates messages from multi-hop neighbors. Geom-GCN [25] aggregates distant but informative node messages through defined geometric relationships. Simp-GCN [12] and UGCN [11] both aggregate messages of distant nodes through the KNN algorithm. Others aim at conducting similarity-based or signed propagation to generate the discriminative node embeddings [2]. GGCN [37] learns signed aggregation weights through edge correction and the cosine function. HOG-GCN [32] measures a node pair's homophily degree by incorporating topological and feature information for aggregation. BM-GCN [10] derives the block similarity matrix to co-guide attribute information propagating and aggregating on network topology. GPR-GNN [3] learns the signed weights through generalized pagerank for aggregation.

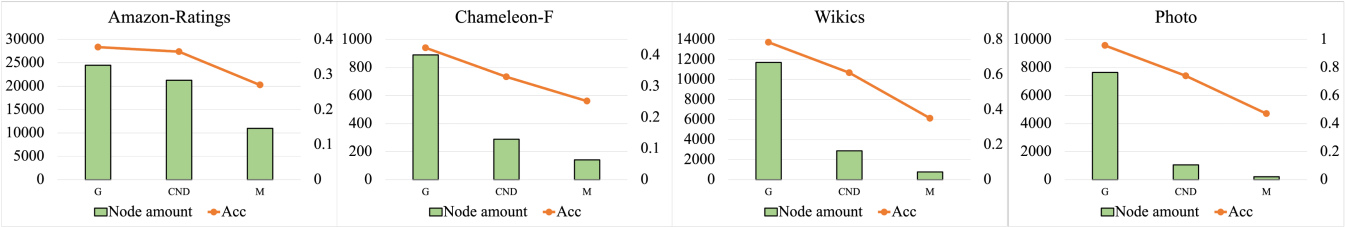



**Figure 2: Conflicting node experiment. The figure shows the amount and prediction accuracy of different node sets in 4 common datasets. G represents the node set of the whole graph. The conflicting node set CND and M are defined in section 4.1.**

CMGNN [41] reformulates the classical methods into a unified heterophilious message-passing (HTMP) mechanism and introduces a new approach that operates within the HTMP mechanism to explicitly leverage and improve the compatibility matrix.

Later work [22, 23] finds that certain types of "good" heterophily exist, under which GCNs can achieve strong performance. Motivated by this observation, a series of works on learning the heterophily pattern come out. ACM-GCN [21] designs an adaptive channel mixing module to learn homophily patterns and heterophily patterns simultaneously. GBK-GNN [5] introduces the bi-kernel feature transformation with a selection gate. It also adaptively learns the homophily and heterophily patterns of each node. Ordered GNN [29] proposes to order the messages passing into the node representation, with specific blocks of neurons targeted for message passing within specific hops. It perfectly embeds the heterophily pattern for later modules to learn and classify. Moreover, some works learn the neighbor distribution to enhance node embeddings. HDP [42] adaptively separates the neighbors into homophilous and heterophilous parts, and both the homophilous and heterophilous patterns are propagated with a semantic-aware message passing mechanism. HA-GAT [31] proposes a graph attention network by fully exploring and utilizing the local distribution as the underlying heterophily to handle networks with different homophily ratios. ND-GCN [17] finds that the neighborhood distribution of nodes with the same class tends to be similar. Based on this observation, they propose a neighborhood distribution-guided graph convolutional network, which can adaptively combine lower-order and higher-order neighborhood distributions. Although the above methods design diverse methods to take advantage of the neighbor distribution, they ignore the phenomenon of conflicting neighbor distributions between different classes, thus resulting in undistinguishable embeddings for nodes with conflicting neighbor distributions. To address this problem, our method designs a conflicting node detection module and a conflicting embedding discrimination module, which aggregate positive or negative messages globally to enhance the conflicting node's discrimination ability.

# 3 Preliminary

## 3.1 Graph Notations

An undirected node graph $G = (V, E, A, X, Y)$ is composed of the node set $V$, edge set $E$, adjacency matrix $A \in R^{|V|\times|V|}$, node feature matrix $X \in R^{|V|\times d}$ and label matrix $Y \in R^{|V|\times C}$, where $C$ is the number of node classes. For each node $v_i \in V$, it has a d-dimensional feature vector $x_i \in X$ and a one-hot label vector $y_i \in Y$. In the semi-supervised node classification task, given the labels of a subset of nodes $V_L$, the goal is to predict the labels of the node set $V \backslash V_L$ through the graph information and the known label set $Y_L$.

## 3.2 Homophily Indicator

The common homophily indicators include node homophily and edge homophily. Node homophily $H_{node}$ represents the average proportion of the neighbors with the same class of each node. Edge homophily $H_{edge}$ represents the average proportion of edges connecting nodes with the same class. The calculation is as follows:

$$H_{node} = \frac{1}{|V|} \sum_{v_i \in V} \frac{|\{v_j \in N(v_i) : y_i = y_j\}|}{|N(v_i)|} \tag{1}$$

$$H_{edge} = \frac{|\{e_{ij} \in E : y_i = y_j\}|}{|E|} \tag{2}$$

where $N(v_i)$ represents the neighbors of $v_i$. The range of node homophily and edge homophily is [0,1]. When the value is close to 1, the graph shows homophily. When the value is close to 0, the graph shows heterophily.

## 3.3 Neighbor Distribution

Neighbor distribution represents the class distribution pattern of one node's neighbors. It describes how a node is connected to different numbers of neighbors with different types of classes, as the case shown in Figure 1. The calculation is as follows:

$$D = AY \tag{3}$$

where $D \in R^{|V|\times C}$ represents the neighbor distribution of all nodes. $D_i$ represents the neighbor distribution of node $v_i$.

# 4 Conflicting Node Experiments and Analysis

Previous works conduct theoretical analyses and design models based on the assumption that nodes of the same class typically exhibit a similar neighbor distribution of node classes in both homophilic and heterophilic graphs. However, our experiments discovered that many nodes belonging to different classes share the same neighbor distribution. The phenomenon of conflicting nodes is universal in the mainstream homophilic and heterophilic graph datasets. The performance of GNN also degrades in the nodes with conflicting neighbor distributions in our experiment.

## 4.1 Key Definitions

This section introduces the key definitions and settings in the experiment. $S(d) = \{v_i \in V | D_i = d\}$ represents the node set sharing the same neighbor distribution $d$. Conflicting nodes union $\mathcal{S} = \{S(d) | d \in D, \exists v_1, v_2 \in S(d), y_1 \neq y_2\}$ describes all of the conflicting node sets whose nodes share the same neighbor distribution but belong to diverse classes (like (b), (c), (e) in Figure 1). $CND = \bigcup_{S(d) \subset \mathcal{S}} S(d)$ includes all of the nodes in each $S(d) \subset \mathcal{S}$ into a large set and it represents all of the conflicting nodes with diverse neighbor distributions. Noticing that most of the nodes in the same conflicting neighbor distribution belong to the same class in homophilic graphs, CND without nodes of the majority class is defined. $S_c(d) = \{v_i | y_i = c, v_i \in S(d)\}(S(d) \in \mathcal{S})$ defines the subset of nodes in $S(d) \in \mathcal{S}$ that belong to class c. Then $M = \bigcup_{S(d) \subset \mathcal{S}} (S(d) - \max_{c \in C} S_c(d))$ represents all of the conflicting nodes that do not belong to the majority class in each conflicting node set (like (e) in Figure 1, where (b) and (c) belong to the majority class). H2GCN [43], which includes common tricks like ego-neighbor separation and representation concatenation, is adopted to evaluate the performance of GNN in nodes of $CND$ and $M$. The accuracy of prediction is notated as $Acc_{CND}$ and $Acc_M$. The experiment is conducted on the homophilic graph dataset (Wikics and Photo) and heterophilic graph dataset (Amazon-Ratings and Chameleon-F). The results are shown in Figure 2.

## 4.2 Empirical Findings

Based on the results, several key observations can be derived:

**1.** Many nodes share the same neighbor distribution as other classes' nodes, either in the homophilic graph datasets or heterophilic graph datasets. The proportion of $M$ is much lower than $CND$, especially in the homophilic graph datasets. It indicates that in the same conflicting neighbor distribution, most of the nodes belong to the same class. The proportion of $M$ is relatively close to $CND$ in the heterophilic graph datasets compared with the homophilic graph datasets. It demonstrates that the classes of nodes in the same conflicting neighbor distribution are more uniform in the heterophilic graph datasets.

**2.** The prediction accuracy of nodes in $CND$ is lower than the accuracy of the whole graph. It is reasonable as the nodes in $CND$ share the same neighbor distribution with the nodes of other classes. So their embeddings are similar and harder to distinguish after graph convolution. The accuracy of nodes in $M$ is lower than that of nodes in $CND$, and the condition is more obvious in homophilic graphs than in heterophilic graphs. The possible reason is that the embeddings of nodes in $CND$ are similar, so the model is more likely to classify the nodes into the majority class because the majority class owns more nodes and contributes more to the cross-entropy loss, thus sacrificing the accuracy of nodes in $M$. As the proportion of $M$ is much lower than $CND$, especially in the homophilic graph datasets, which means the majority class gains greater dominance in the homophilic datasets, so the accuracy of $M$ in homophilic graph datasets is relatively lower compared with heterophilic graph datasets.

## 4.3 Theoretical Analysis

The degradation of GNN's performance in nodes of conflicting neighbor distribution is consistent with previous theorem in [23]. It proves the theorem as follows.

**Theorem 1.** Consider a graph $G \sim CSBM(\mu_1, \mu_2, p, q)$. For any node $i$ in this graph, the linear classifier defined by the decision boundary $P$ has a lower probability to misclassify $h_i$ than $x_i$ when $deg(i) > (p+q)^2/(p-q)^2$.

where $\mu_1, \mu_2$ are the mean value of the initial feature $x_1, x_2$. Any two nodes in the graph are connected by an edge with probability $p$ if they are from the same class; otherwise, the probability is $q$. Specifically, the neighbor distributions corresponding to class 1 and class 2 are $[\frac{p}{p+q}, \frac{q}{p+q}]$, $[\frac{q}{p+q}, \frac{p}{p+q}]$, respectively. When $p$ is equal to $q$, nodes of the two classes share the same neighbor distribution. In this case, $(p+q)^2/(p-q)^2$ tends towards positive infinity and $deg(i) > (p+q)^2/(p-q)^2$ is impossible to meet. So the performance of GNN degrades and is inferior to MLP, which is consistent with our experiment's conclusion theoretically.

## 4.4 Motivation

To improve the prediction accuracy of nodes with conflicting neighbor distribution, the key objective is to increase the discrimination of node embeddings with the same neighbor distribution. The idea is that, for conflicting nodes with the same neighbor distribution, we try to predict their class type based on some indicators and then aggregate conflicting nodes of the same predicted class type using positive weights and nodes of different predicted class types using negative weights. The estimation of conflicting nodes' class types can be achieved with two indicators. **1. High-order neighbor distribution.** Although these nodes are conflicting in the 1-hop neighbor distribution, they are more discriminative in the high-order neighbor distribution. The statistical data demonstrates that the 2nd-order neighbor distribution of the conflicting nodes shows higher similarity between the same class than different classes. **2. Original node feature.** Theorem 1 illustrates that the original node features are more discriminative than features after GNN when the neighbor distribution is conflicting. So the original node features can be a discriminative indicator for the conflicting nodes.

# 5 Method

The method is designed to detect the possible conflicting nodes and enhance the discrimination of the conflicting node embeddings to improve the prediction accuracy of semi-supervised node classification on both homophilic and heterophilic graphs. The framework is shown in Figure 3, and the description of each module is as follows.

## 5.1 Conflicting Node Detection

The conflicting node detection module aims at estimating each node's neighbor distribution according to the high-quality pseudo-label and detecting the possible conflicting nodes with the same neighbor distribution.

The neighbor distribution of each node is uncertain, as only a small part of labels are available in the semi-supervised task, so the module initially predicts all of the nodes' labels through GCN, MLP or other baselines. The formula is as follows.

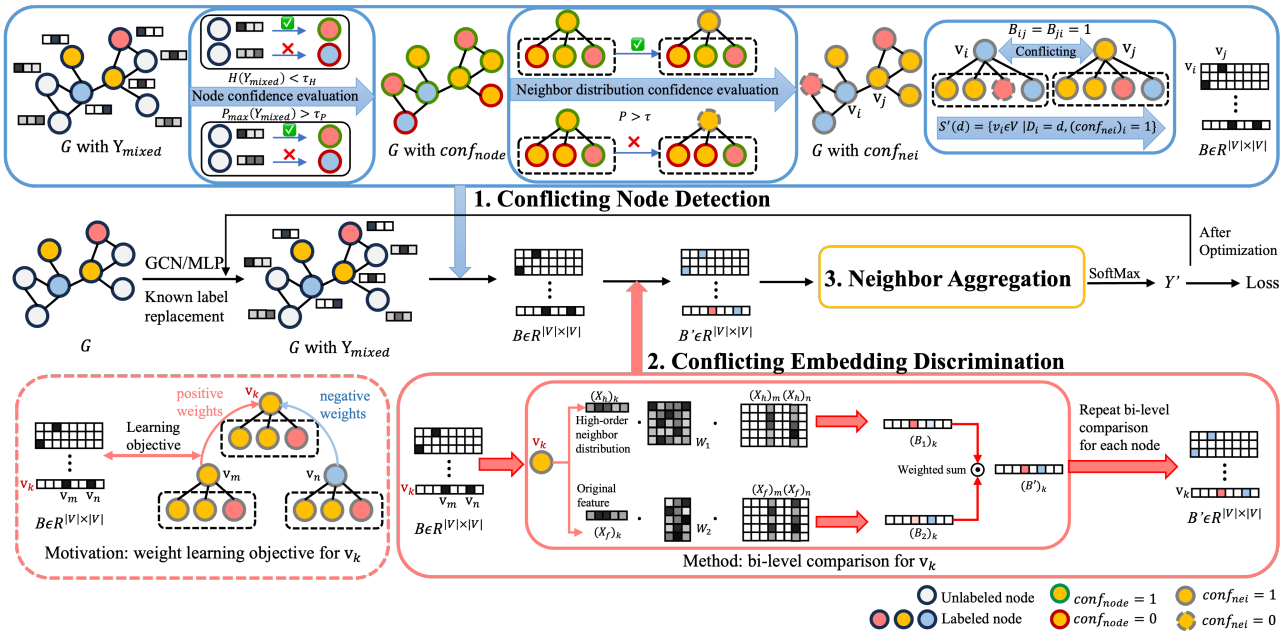



**Figure 3: Framework of CNDGNN. The figure shows the pipeline of our method. The proposed conflicting node detection module and conflicting embedding discrimination module are described in detail.**

$$Y_{pse} = Softmax(f(X)) \tag{4}$$

where $f(.)$ represents baseline methods. $Y_{pse} \in R^{|V|\times C}$ represents each node's probability distribution of class type. For labeled nodes in $Y_{pse}$, their probability distributions are replaced with their one-hot codes of labels to generate the mixed labels $Y_{mixed}$.

$$Y_{mixed} = MY + (I - M)Y_{pse}, M = \text{diag}(mask) \tag{5}$$

where $Y$ is the one-hot matrix of known labels, diag(.) means the diagonal matrix, mask $\in R^{|V|}$ describes the nodes with known labels, $mask_i = 1$ represents the label of $v_i$ is known, otherwise $mask_i = 0$. Except for the soft pseudo-label $Y_{mixed}$, the one-hot pseudo-label matrix $D_{mixed} \in R^{|V|\times C}$ is generated by selecting the maximum probability value in $Y_{mixed}$ as the class type of each node. So the neighbor distribution of each node is as follows.

$$D = AD_{mixed} \tag{6}$$

With each node's probability distribution of class type, the confidence of its pseudo-label is evaluated to select the node whose class type is relatively certain. The node confidence matrix $conf_{node} \in R^{|V|}$ is evaluated by the indicator function as follows.

$$conf_{node} = \mathbf{1}[\, H(Y_{mixed}) < \tau_H \,] \odot \mathbf{1}[\, P_{\max}(Y_{mixed}) > \tau_P \,] \tag{7}$$

where $\mathbf{1}[.]$ is the indicator function, $\odot$ means element-wise production. $H(Y_{mixed}) \in R^{|V|}$ is the entropy matrix of each node in $Y_{mixed}$. $P_{\max}(Y_{mixed}) \in R^{|V|}$ describes maximum probability value of each node in $Y_{mixed}$. $\tau_H$ and $\tau_P$ are threshold values. Usually, Smaller entropy means higher confidence in the class type. While only a small entropy is not enough to decide the probability distribution's confidence when the probability distribution is $[0.5, 0.5, 0, \ldots, 0]$, where the entropy is also small, but the model is entangled between two classes. So $P_{\max}(Y_{mixed})$ is introduced to make sure $y_{mixed}$ is certain with only one class type. After the indicator function, the labeled nodes and unlabeled nodes with high confidence will be set as 1 in $conf_{node}$, and these nodes are the high-quality nodes.

With each node's confidence about its label, the confidence of the node's neighbor distribution $conf_{nei} \in R^{|V|}$ is evaluated as follows.

$$conf_{nei} = \mathbf{1}[\, P > \tau \,], P = (A \cdot conf_{node}) \oslash Deg \tag{8}$$

where $\oslash$ means element-wise division, $Deg \in R^{|V|}$ is the degree matrix, and $\tau$ is the threshold value. $P$ calculates the proportion of confident nodes in one node's neighbor distribution. With the indicator function, only nodes whose neighbor distribution owns a high proportion of confident nodes are set as 1 in $conf_{nei}$.

Attaining each node's neighbor distribution $D$ (Eq.6) and its confidence $conf_{nei}$ (Eq.8), the detection of conflicting nodes is consistent with section 4.1. $S'(d) = \{v_i \in V | D_i = d, (conf_{nei})_i = 1\}$ represents the node set sharing the same neighbor distribution and owning a highly confident neighbor distribution. Conflicting nodes union $\mathcal{S}' = \{S'(d) | d \in D, \exists v_1, v_2 \in S'(d), y_1 \neq y_2\}$ describes all of the sets whose nodes share the same neighbor distribution but belong to diverse classes. The conflicting adjacency matrix $B \in R^{|V|\times|V|}$ is defined as follows.

$$B_{ij} = B_{ji} = 1(v_i, v_j \in S'(d), S'(d) \subset \mathcal{S}') \tag{9}$$

$B$ describes the connection relationship of all conflicting nodes belonging to the same neighbor distribution.

## 5.2 Conflicting Embedding Discrimination

The conflicting embedding discrimination module aims at increasing the discrimination of node embeddings with the same neighbor distribution. It estimates the class type of the nodes with the same neighbor distribution through the node's high-order neighbor distribution and original feature, and a weight matrix is learned to assign positive weights for nodes of the same class type and negative weights for nodes of different types for later aggregation.

From the perspective of high-order neighbor distribution, the method chooses 2-order neighbor distribution as it maintains both a large inter-class distance and a small intra-class distance. The high-order neighbor distribution is as follows.

$$D_h = A^2 D_{mixed} \tag{10}$$

The high-order neighbor distribution of the conflicting nodes shows higher similarity between the same class than between different classes. So the module compares the similarity of conflicting nodes' high-order neighbor distributions, and assigns positive weights to nodes of higher similarity (assumed as the same class), and negative weights to nodes of lower similarity (assumed as different classes). The weight matrix $B_1 \in R^{|V|\times|V|}$ is generated as follows.

$$B_1 = \tanh\left(X_h W X_h^{\mathsf{T}}\right), X_h = MLP(D_h) \tag{11}$$

where $W$ is the parameter matrix. Here, the model firstly conducts a linear transformation to $D_h$ to enhance its expressive power. Then the production with parameter matrix $W$ is applied to measure the similarity between node pairs. Finally, the activation function tanh is adopted to output the positive or negative weights ranging from $-1$ to 1 according to the similarity value.

From the perspective of the original node feature, the weight matrix $B_2 \in R^{|V|\times|V|}$ is generated similarly to $B_1$ as follows.

$$B_2 = \tanh\left(X_f W X_f^{\mathsf{T}}\right), X_f = MLP(X) \tag{12}$$

where $X$ is the original node features. The activation function tanh is also adopted to output the positive or negative weights.

Finally, the weight matrices $B_1$ and $B_2$ are incorporated through a weighted sum function as follows.

$$B' = B \odot (W' B_1 + (I - W') B_2), W' = diag(Sigmoid(W_B)) \tag{13}$$

where $\odot$ represents element-wise production, $W_B \in R^{|V|}$ is the learnable incorporation parameter for each node. The parameter matrix $W_B$ is activated by the *Sigmoid* function to generate weights ranging from 0 to 1 for nodes to incorporate. Then the incorporated matrix conducts element-wise production with the conflicting adjacency matrix $B$ to make sure that the learned weight matrix influences only the conflicting nodes.

## 5.3 Neighbor Aggregation

The neighbor aggregation module aggregates ego feature, neighbor feature, and conflicting embedding through common tricks like ego-neighbor separation and layer concatenation consistent with previous methods [21, 23, 41].

The ego feature $Z_1^l$, neighbor feature $Z_2^l$, and conflicting embedding $Z_3^l$ are constructed as follows.

$$Z_1^l = \mathrm{ReLU}\left(Z_1^{(l-1)} W_1^l\right), Z_2^l = \mathrm{ReLU}\left(\mathrm{diag}(\mathrm{Deg}^{-1}) A Z_2^{(l-1)} W_2^l\right) \tag{14}$$

$$Z_3^l = \tanh\left(B' Z_3^{(l-1)} W_3^l\right) \tag{15}$$

Then these embeddings are aggregated through learnable weights as follows.

$$\begin{aligned} [\alpha_1^l, \alpha_2^l, \alpha_3^l] &= \mathrm{Softmax}\left(\sigma\left([Z_1^l, Z_2^l, Z_3^l]\tilde{W}\right)\right) \in R^{|V|\times 3} \\ Z^l &= \mathrm{diag}(\alpha_1^l) Z_1^l + \mathrm{diag}(\alpha_2^l) Z_2^l + \mathrm{diag}(\alpha_3^l) Z_3^l \end{aligned} \tag{16}$$

Where $Z^l$ is the final node embedding of the $l$-th layer. The final prediction of class type is conducted on the concatenated layers.

$$Y' = Softmax(W(\Big\|_{l=0}^{L} Z^l)) \tag{17}$$

where $W$ is the parameter matrix. The cross-entropy loss function is adopted to optimize parameters based on the prediction $Y'$. As the optimized $Y'$ after a complete training process achieves higher prediction accuracy, we apply the optimized high-quality pseudo-labels as the input of our method and iteratively train several times.

## 5.4 Theoretical Analysis

Consistent with Theorem 1, the theoretical analysis of our method is as follows.

**Theorem 2.** Consider a graph $G \sim CSBM(\mu_1, \mu_2, p, q)$, the amount of added conflicting nodes is $\alpha$ times that of each node's degree, and the accuracy of estimating the conflicting node's class type (the same class or different classes) is $e$. For any node $i$ in this graph, the linear classifier defined by the decision boundary $P$ has a lower probability to misclassify $h_i$ of CNDGNN than $x_i$ when $deg(i) > \frac{1+\alpha}{[\frac{p-q}{p+q}+\alpha(2e-1)]^2}$.

The detailed proof and analysis are available in Appendix A. Based on the analysis, we conclude that our method performs better in the conflicting nodes compared with GCN in Theorem 1.

# 6 Experiments

In this section, comprehensive experiments are conducted to evaluate the semi-supervised node classification performance of CNDGNN in both homopholic and heterophilic datasets. The experiments aim at answering the following questions.

**Q1.** How does the proposed CNDGNN perform in both homopholic and heterophilic datasets?

**Q2.** Are the proposed modules in CNDGNN effective?

**Q3.** How does CNDGNN perform in conflicting nodes compared with other SOTA methods? Can the proposed modules improve the prediction accuracy in conflicting nodes?

**Q4.** What's the computation complexity of CNDGNN?

**Q5.** Is CNDGNN sensitive about parameters?

## 6.1 Dataset and Settings

As is pointed out in [26], there are serious data leakages in Chameleon and Squirrel, while Cornell, Texas, and Wisconsin are too small with very imbalanced classes. Here, we adopt 8 representative or optimized datasets with various heterophily levels to better evaluate

**Table 1: Main results. It describes the node classification accuracy of each method. The best result is bold and the second-best result is underlined. OOM denotes out-of-memory error during model training.**

| Dataset | Roman-Empire | Amazon-Ratings | Chameleon-F | Squirrel-F | Flickr | Wikics | Pubmed | Photo |
|---|---|---|---|---|---|---|---|---|
| Homophily | 0.05 | 0.38 | 0.25 | 0.22 | 0.24 | 0.65 | 0.8 | 0.83 |
| Nodes | 22662 | 24492 | 890 | 2223 | 7575 | 11701 | 19717 | 7650 |
| Edges | 65854 | 186100 | 13584 | 65718 | 479476 | 431206 | 88651 | 238162 |
| Classes | 18 | 5 | 5 | 5 | 9 | 10 | 3 | 8 |
| MLP | 62.19 ± 1.03 | 42.66 ± 0.84 | 38.66 ± 4.02 | 36.74 ± 1.80 | 89.82 ± 0.63 | 78.94 ± 1.22 | 87.48 ± 0.46 | 89.96 ± 1.22 |
| GCN | 38.58 ± 2.35 | 45.16 ± 0.49 | 42.12 ± 3.82 | 38.47 ± 1.82 | 68.25 ± 2.75 | 77.53 ± 1.41 | 87.70 ± 1.32 | 94.31 ± 0.33 |
| GAT | 59.55 ± 1.45 | 46.90 ± 0.47 | 40.89 ± 3.50 | 38.22 ± 1.71 | 57.22 ± 3.04 | 76.69 ± 0.87 | 87.45 ± 0.53 | 94.59 ± 0.48 |
| H2GCN | 68.61 ± 1.05 | 37.20 ± 0.67 | 42.29 ± 4.57 | 35.82 ± 2.20 | 91.25 ± 0.58 | 78.34 ± 2.01 | 89.32 ± 0.37 | 95.66 ± 0.26 |
| MixHop | 79.16 ± 0.70 | 47.95 ± 0.65 | 44.97 ± 3.12 | 40.43 ± 1.40 | 91.10 ± 0.46 | 84.19 ± 0.61 | 89.42 ± 0.37 | 95.63 ± 0.30 |
| GBK-GNN | 66.05 ± 1.44 | 40.20 ± 1.96 | 42.01 ± 4.89 | 36.52 ± 1.45 | OOM | 81.07 ± 0.83 | 88.18 ± 0.45 | 93.48 ± 0.42 |
| GGCN | OOM | OOM | 41.23 ± 4.08 | 36.76 ± 2.19 | 90.84 ± 0.65 | 84.76 ± 0.65 | 89.04 ± 0.40 | 95.18 ± 0.44 |
| GloGNN | 68.63 ± 0.63 | 48.62 ± 0.59 | 40.95 ± 5.95 | 36.85 ± 1.97 | 90.47 ± 0.77 | 82.83 ± 0.52 | 89.60 ± 0.34 | 95.09 ± 0.46 |
| HOG-GCN | OOM | OOM | 43.35 ± 3.66 | 38.63 ± 1.95 | 90.94 ± 0.72 | 83.74 ± 0.69 | OOM | 94.79 ± 0.26 |
| GPRGNN | 71.19 ± 0.75 | 46.64 ± 0.52 | 41.84 ± 4.68 | 38.04 ± 1.98 | 91.19 ± 0.47 | 84.47 ± 0.54 | 89.28 ± 0.37 | 95.48 ± 0.24 |
| ACM-GCN | 71.15 ± 0.73 | 50.58 ± 0.62 | 44.30 ± 3.57 | **40.90 ± 1.74** | 91.43 ± 0.65 | 84.39 ± 0.43 | **89.99 ± 0.40** | 95.52 ± 0.40 |
| DGAT | 79.34 ± 4.62 | 50.17 ± 0.93 | 41.62 ± 3.58 | 40.31 ± 1.86 | 90.18 ± 0.88 | 82.49 ± 0.74 | 88.55 ± 0.31 | 94.72 ± 0.75 |
| LHKGNN | 65.61 ± 0.77 | 42.67 ± 0.53 | 41.61 ± 4.77 | 37.21 ± 2.12 | 90.60 ± 0.57 | 82.99 ± 0.93 | 88.58 ± 0.35 | 94.37 ± 0.39 |
| CNDGNN$_{GCN}$ | 81.32 ± 0.71 | 51.79 ± 0.63 | **45.67 ± 2.93** | 39.39 ± 2.17 | 92.04 ± 0.72 | 85.62 ± 0.82 | 89.38 ± 0.37 | 95.77 ± 0.43 |
| CNDGNN$_{GPRGNN}$ | **83.13 ± 0.62** | **51.83 ± 0.68** | 45.62 ± 3.08 | 39.42 ± 2.09 | **92.09 ± 0.64** | **85.87 ± 0.86** | 89.43 ± 0.35 | **95.79 ± 0.31** |

methods' performance [20, 41]. It includes 5 heterophilic datasets (Roman-Empire, Amazon-Ratings, Chameleon-F, Squirrel-F, and Flickr) and 3 homophilic datasets (Wikics, Pubmed [28, 39] and Photo). They all include diverse amounts of conflicting nodes. The detailed statistics of each dataset are described in Table 1. More details about each dataset are available in Appendix B.

The experiments are conducted on one Nvidia RTX 4060ti with the memory of 16GB. Consistent with previous methods [41], the proportions of training, validation, and testing sets are 48%,32%, and 20%, respectively. All of the methods are trained with the same optimizer Adam, and the same random seed set. An early stop strategy is applied for model training with the parameter "patience". For each method, we use the official code released by the corresponding authors and finetune the parameters as suggested in the original paper. The results report the mean performance and standard deviation of 10 splits.

## 6.2 Baseline Methods

We compare CNDGNN with a series of baseline methods, including: (1) a graph-agnostic method: MLP. MLP only uses the node's feature for classification. (2) classical GNNs: GCN [13], GAT [30]. These methods adopt message passing for node embedding and do not design modules to adapt to heterophilic graphs. (3) methods adapted to heterophilous graphs: H2GCN [43], MixHop [1], GBK-GNN [5], GGCN [37], GloGNN [15], HOG-GCN [32], GPRGNN [3], ACM-GCN [21], DGAT [19], LHKGNN [27]. These methods design diverse tricks, like high-order neighbor aggregation, similarity-based aggregation, and signed propagation to adapt to heterophilic graphs.

## 6.3 Main Result (Q1)

Table 1 shows the semi-supervised node classification performance results of all the methods on 8 benchmark datasets. The subscript in CNDGNN represents the initialization baselines in Eq.4. Here we select GCN and GPRGNN.

For the classical methods GCN and GAT, they achieve strong performance on highly homophilous datasets. However, their performance drops dramatically on heterophilous datasets, often performing worse than a simple MLP. This confirms that naively aggregating features from neighbors can result in undiscriminating embeddings in heterophilous settings. For models adapted to heterophilous graphs, such as MixHop, GPRGNN, and GGCN, they consistently outperform GCN/GAT on heterophilous datasets. By incorporating information from higher-order neighborhoods or signed propagation, they alleviate the misleading signals from immediate neighbors. Nevertheless, these approaches often suffer from scalability issues in large-scale datasets. For methods learning heterophily patterns, including ACM-GCN and our proposed CNDGNN, they demonstrate superior robustness across different levels of homophily. ACM-GCN achieves state-of-the-art performance on Pubmed, where homophily is high, while maintaining competitive results in other settings. Our CNDGNN achieves the best or near-best performance on almost all datasets. These results highlight the effectiveness of detecting and discriminating conflicting nodes, which enables CNDGNN to adapt to both homophilous and heterophilous graphs and enhances the prediction accuracy of conflicting nodes.

Moreover, different initialization baselines influence the results a little. For Roman-Empire, the initialization baseline GCN performs much worse than GPRGNN, while the result only drops a little

**Table 2: Ablation results. It shows the node classification accuracy of CNDGNN$_{GCN}$ without the conflicting node detection module, the conflicting embedding discrimination module and circle training structure.**

| Dataset | Roman-Empire | Amazon-Ratings | Chameleon-F | Squirrel-F | Flickr | Wikics | Pubmed | Photo |
|---|---|---|---|---|---|---|---|---|
| CNDGNN-circle | 77.11 ± 1.51 | 49.87 ± 0.79 | 42.36 ± 2.92 | 37.28 ± 2.10 | 90.64 ± 0.76 | 80.71 ± 3.49 | 87.75 ± 0.70 | 93.01 ± 1.56 |
| CNDGNN-CND | 70.15 ± 1.56 | 50.05 ± 0.76 | 44.27 ± 3.96 | 38.27 ± 2.07 | 90.79 ± 0.71 | 82.53 ± 0.80 | 87.72 ± 0.63 | 94.23 ± 0.42 |
| CNDGNN-CED | 79.42 ± 1.38 | 48.26 ± 1.13 | 42.02 ± 3.69 | 37.89 ± 2.21 | 90.28 ± 0.78 | 83.89 ± 0.94 | 86.38 ± 1.06 | 94.67 ± 0.32 |
| CNDGNN$_{GCN}$ | **81.32 ± 0.71** | **51.79 ± 0.63** | **45.67 ± 2.93** | **39.39 ± 2.17** | **92.04 ± 0.72** | **85.62 ± 0.82** | **89.38 ± 0.37** | **95.77 ± 0.43** |

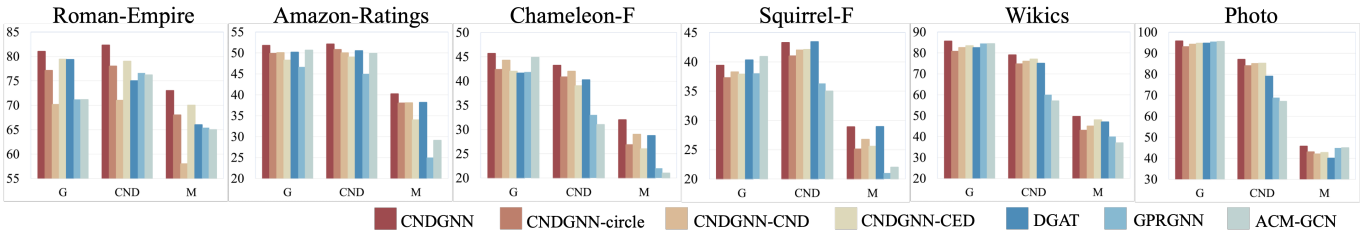



**Figure 4: Performance in conflicting nodes. The figure shows the prediction accuracy of our methods and other methods in conflicting nodes. G represents the node set of the whole graph. The conflicting node set CND and M are defined in section 4.1.**

and remains competitive. Although stronger initialization baselines help boost performance, we think CNDGNN is not very sensitive to the quality of pseudo-labels. GPRGNN also provides more precise pseudo-labels in the Flickr dataset, while CNDGNN enjoys less improvement due to the quite low proportion of conflicting nodes.

## 6.4 Ablation Study (Q2)

This section tests the effectiveness of the proposed key modules. We remove the circle training structure (CNDGNN-circle), the conflicting node detection module (CNDGNN-CND), and the conflicting embedding discrimination module (CNDGNN-CED), respectively. The performance of each model is shown in Table 2.

In general, the performance of each ablated model is worse than CNDGNN$_{GCN}$. As the proportion of conflicting nodes in the dataset Flickr is lower than in other datasets, the removal of modules influences it less than in other datasets. For the CNDGNN-circle model, it removes the circle training structure, so it only detects the possible conflicting nodes based on the pseudo-labels predicted from basic GCN, and wastes the relatively high-quality pseudo-labels from the whole model, thus decreasing the amount of the conflicting nodes with high confidence and resulting in a low-quality conflicting node matrix. For the CNDGNN-CND model, as it removes the conflicting node detection module, the model includes all of the possible conflicting nodes without considering the confidence of pseudo-labels, so some of the ambiguous nodes are included in the conflicting nodes, which will mislead the latter module and degrade performance. For CNDGNN-CED, it removes the conflicting embedding discrimination module, so the signed weight matrix is replaced with a positive weight matrix, which will assign the positive weights to conflicting nodes from different classes, thus decreasing the prediction accuracy. Moreover, the removal of key modules also makes the variance of accuracy larger in some datasets. The possible reason is that the low-quality evaluation of conflicting nodes brings about the ambiguous neighbor distribution and thus results in unstable accuracy.

## 6.5 Performance in Conflicting Nodes (Q3)

In this section, we compare the performance of different models in the whole graph and the conflicting nodes set CND, and M. Except for CNDGNN$_{GCN}$, we also choose the ablated model CNDGNN-circle, CNDGNN-CND, and CNDGNN-CED. The SOTA models, including DGAT, GPRGNN, and ACM-GCN, are included for comparison. The result is shown in Figure 4.

It can be seen from the figure that CNDGNN achieves the best performance in the conflicting nodes of diverse datasets. It demonstrates CNDGNN's advantage of detecting and discriminating conflicting nodes, thus improving the prediction accuracy of the whole graph. It is noticeable that for dataset Squirrel-F, although the accuracy of CNDGNN in the whole graph is lower than DGAT and ACM-GCN, CNDGNN achieves comparable or higher accuracy in the conflicting node datasets.

For the ablated models, their prediction accuracy in the conflicting nodes is lower than CNDGNN to different extents due to the lack of diverse key modules. As they still reserve some modules dealing with conflicting nodes, their accuracy in conflicting nodes is generally higher than other SOTA models. For the SOTA models, as they do not specifically optimize for conflicting nodes, their accuracy in conflicting node sets CND and M is relatively lower. Between these models, DGAT equips the attention mechanism with parameterized edge features to adapt to the specific characteristics of the global topology, which is similar to our weight matrix learning according to the high-order neighbors, and also achieves great performance in conflicting nodes.

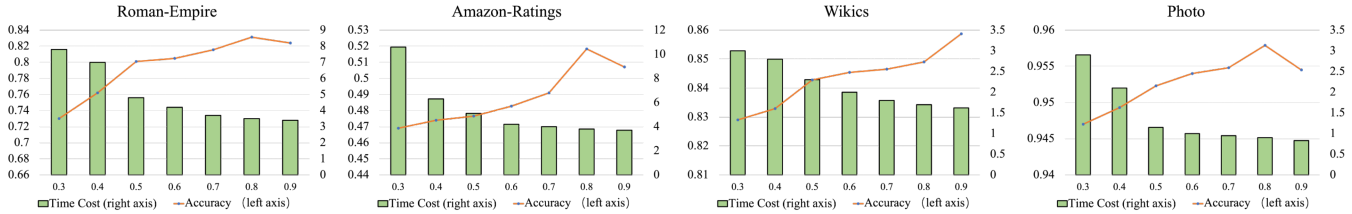



**Figure 5: Parameter Sensitivity. The figure shows the prediction accuracy and training time (s) per 10 epochs of CNDGNN$_{GPRGNN}$ when the threshold value $\tau$ in Eq.8 changes from 0.3 to 0.9.**

**Table 3: Training time (s) per 10 epochs of different methods in large datasets.**

| Dataset | Roman-Empire | Amazon-Ratings | Pubmed |
|---|---|---|---|
| CNDGNN$_{GPRGNN}$ | 3.5 | 3.8 | 6.2 |
| MixHop | 29.8 | 35.2 | 4.3 |
| GBKGNN | 7.9 | 11.3 | 7.2 |
| LHKGNN | 5.6 | 7.1 | 4.2 |

## 6.6 Complexity Analysis (Q4)

Let $N_c$ represent the number of all conflicting nodes; $K$ represents the category number of conflicting neighbor distributions. The aggregation complexity of conflicting node messages is $O(N_c^2/K)$. When $K$ is close to $N_c$, the complexity is close to $O(N_c)$. The aggregation of conflicting messages is efficiently accomplished by python's sparse matrix multiplication, which makes sure that only conflicting nodes' aggregation weights are calculated in Eq.11 and Eq.12 to boost the aggregation process. Based on the analysis, the actual complexity depends on the concrete value of $N_c$ and $K$. It's a trade-off, as more conflicting nodes with neighbor distributions of high discriminative power bring more improvement and also cost more time. Moreover, it's noticeable that the conflicting node detecting module filters out low-quality pseudo-labels, thus reducing the potential conflicting node amount $N_c$ at the same time, which not only improves the accuracy, but also helps reduce the time complexity.

We report the training time per 10 epochs of CNDGNN$_{GPRGNN}$ (CNDGNN initialized with GPRGNN), MixHop [1], GBKGNN [5], LHKGNN[27] in the large datasets: Roman-Empire, Amazon-Ratings and Pubmed. Mixhop is a representative method that aggregates high-order nodes. GBKGNN selectively aggregates neighbor nodes. LHKGNN is a spectrum-based method. The results are available in table 3. Compared with MixHop, which also aggregates high-order nodes, our method costs much less time in Roman-Empire and Amazon-Ratings, and the training time of our method is competitive compared with GBKGNN and LHKGNN in most of the large datasets.

## 6.7 Parameter Sensitivity (Q5)

The sensitivity of the threshold value $\tau$ in Eq.8 is analyzed in this section. We change the value of $\tau$ from 0.3 to 0.9 and report the accuracy and training time (s) per 10 epochs of CNDGNN$_{GPRGNN}$. The results are shown in Figure 5.

As $\tau$ increases, the conflicting node detection module filters out more low-quality pseudo-labels, which not only reduces the conflicting node amount, but also improves the quality of pseudo-labels for the latter prediction. So, it can be seen from Figure 5 that as $\tau$ increases, the training time drops quickly and the accuracy increases in general. The most suitable value of $\tau$ is around 0.8. When the value of $\tau$ increases to 0.9, it may result in the filtration of some high-quality pseudo-labels in some datasets and make the accuracy drop a little. Moreover, when the value of $\tau$ ranges from 0.5 to 0.9, CNDGNN remains competitive between baselines, which shows that CNDGNN is competitive when the value of $\tau$ is set appropriately.

## 7 Conclusion

In this paper, we find the phenomenon that many nodes belonging to different classes share the same neighbor distribution in the mainstream graph datasets. The performance of GNN degrades in these conflicting nodes and drags the whole prediction accuracy down. To tackle the challenge of conflicting nodes, we propose a new method named Conflicting Node Discrimination GNN (CNDGNN). It includes two key modules. The conflicting node detection module predicts each node's neighbor distribution according to the high-quality pseudo-label judged by the prediction confidence, and detects the possible conflicting nodes belonging to the same neighbor distribution. The conflicting embedding discrimination module estimates the class type of the nodes with the same neighbor distribution through the node's high-order neighbor distribution and original feature, and a weight matrix is learned to assign positive weights for nodes of the same class type and negative weights for nodes of different types for later aggregation. Moreover, we theoretically analyze our method through the CSBM model and prove that it performs better than GCN in the conflicting nodes. The theorem analysis also explores the key design view when solving the conflicting node problem. In the future, we will explore how to better estimate the class type of conflicting nodes to increase the estimation accuracy. New types of propagation strategies can also be applied to the conflicting embedding discrimination module to improve its discriminating ability.

## Acknowledgments

This work is partially supported by NSFC (No. 62472068).

## References


[1] Sami Abu-El-Haija, Bryan Perozzi, Amol Kapoor, Nazanin Alipourfard, Kristina Lerman, Hrayr Harutyunyan, Greg Ver Steeg, and Aram Galstyan. 2019. Mixhop: Higher-order graph convolutional architectures via sparsified neighborhood mixing. In *international conference on machine learning*. PMLR, 21–29.

[2] Deyu Bo, Xiao Wang, Chuan Shi, and Huawei Shen. 2021. Beyond low-frequency information in graph convolutional networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 3950–3957.

[3] Eli Chien, Jianhao Peng, Pan Li, and Olgica Milenkovic. 2020. Adaptive universal generalized pagerank graph neural network. *arXiv preprint arXiv:2006.07988* (2020).

[4] Valerio Ciotti, Moreno Bonaventura, Vincenzo Nicosia, Pietro Panzarasa, and Vito Latora. 2016. Homophily and missing links in citation networks. *EPJ Data Science* 5, 1 (2016), 7.

[5] Lun Du, Xiaozhou Shi, Qiang Fu, Xiaojun Ma, Hengyu Liu, Shi Han, and Dongmei Zhang. 2022. Gbk-gnn: Gated bi-kernel graph neural networks for modeling both homophily and heterophily. In *Proceedings of the ACM web conference 2022*. 1550–1558.

[6] Chen Gao, Xiang Wang, Xiangnan He, and Yong Li. 2022. Graph neural networks for recommender system. In *Proceedings of the fifteenth ACM international conference on web search and data mining*. 1623–1625.

[7] Chen Gao, Yu Zheng, Nian Li, Yinfeng Li, Yingrong Qin, Jinghua Piao, Yuhan Quan, Jianxin Chang, Depeng Jin, Xiangnan He, et al. 2023. A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems* 1, 1 (2023), 1–51.

[8] Tao Guo and Baojiang Cui. 2021. Web page classification based on graph neural network. In *International Conference on Innovative Mobile and Internet Services in Ubiquitous Computing*. Springer, 188–198.

[9] Yu Hao, Xin Cao, Yufan Sheng, Yixiang Fang, and Wei Wang. 2021. Ks-gnn: Keywords search over incomplete graphs via graphs neural network. *Advances in Neural Information Processing Systems* 34 (2021), 1700–1712.

[10] Dongxiao He, Chundong Liang, Huixin Liu, Mingxiang Wen, Pengfei Jiao, and Zhiyong Feng. 2022. Block modeling-guided graph convolutional neural networks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 4022–4029.

[11] Di Jin, Zhizhi Yu, Cuiying Huo, Rui Wang, Xiao Wang, Dongxiao He, and Jiawei Han. 2021. Universal graph convolutional networks. *Advances in neural information processing systems* 34 (2021), 10654–10664.

[12] Wei Jin, Tyler Derr, Yiqi Wang, Yao Ma, Zitao Liu, and Jiliang Tang. 2021. Node similarity preserving graph convolutional networks. In *Proceedings of the 14th ACM international conference on web search and data mining*. 148–156.

[13] TN Kipf. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations*.

[14] Chaozhuo Li, Bochen Pang, Yuming Liu, Hao Sun, Zheng Liu, Xing Xie, Tianqi Yang, Yanling Cui, Liangjie Zhang, and Qi Zhang. 2021. Adsgnn: Behavior-graph augmented relevance modeling in sponsored search. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 223–232.

[15] Xiang Li, Renyu Zhu, Yao Cheng, Caihua Shan, Siqiang Luo, Dongsheng Li, and Weining Qian. 2022. Finding global homophily in graph neural networks when meeting heterophily. In *International conference on machine learning*. PMLR, 13242–13256.

[16] Meng Liu, Zhengyang Wang, and Shuiwang Ji. 2021. Non-local graph neural networks. *IEEE transactions on pattern analysis and machine intelligence* 44, 12 (2021), 10270–10276.

[17] Siqi Liu, Dongxiao He, Zhizhi Yu, Di Jin, and Zhiyong Feng. 2025. Beyond homophily: Neighborhood distribution-guided graph convolutional networks. *Expert Systems with Applications* 259 (2025), 125274.

[18] Zhiwei Liu, Liangwei Yang, Ziwei Fan, Hao Peng, and Philip S Yu. 2022. Federated social recommendation with graph neural network. *ACM Transactions on Intelligent Systems and Technology (TIST)* 13, 4 (2022), 1–24.

[19] Qincheng Lu, Jiaqi Zhu, Sitao Luan, and Xiao-Wen Chang. 2024. Flexible Diffusion Scopes with Parameterized Laplacian for Heterophilic Graph Learning. In *The Third Learning on Graphs Conference*.

[20] Sitao Luan, Chenqing Hua, Qincheng Lu, Liheng Ma, Lirong Wu, Xinyu Wang, Minkai Xu, Xiao-Wen Chang, Doina Precup, Rex Ying, et al. 2024. The heterophilic graph learning handbook: Benchmarks, models, theoretical analysis, applications and challenges. *arXiv preprint arXiv:2407.09618* (2024).

[21] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Mingde Zhao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. 2022. Revisiting heterophily for graph neural networks. *Advances in neural information processing systems* 35 (2022), 1362–1375.

[22] Sitao Luan, Chenqing Hua, Minkai Xu, Qincheng Lu, Jiaqi Zhu, Xiao-Wen Chang, Jie Fu, Jure Leskovec, and Doina Precup. 2023. When do graph neural networks help with node classification? investigating the homophily principle on node distinguishability. *Advances in Neural Information Processing Systems* 36 (2023), 28748–28760.

[23] Yao Ma, Xiaorui Liu, Neil Shah, and Jiliang Tang. 2022. Is Homophily a Necessity for Graph Neural Networks?. In *International Conference on Learning Representations*.

[24] Jinyoung Park, Sungdong Yoo, Jihwan Park, and Hyunwoo J Kim. 2022. Deformable graph convolutional networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 7949–7956.

[25] Hongbin Pei, Bingzhe Wei, Kevin Chen-Chuan Chang, Yu Lei, and Bo Yang. 2020. Geom-gcn: Geometric graph convolutional networks. *arXiv preprint arXiv:2002.05287* (2020).

[26] Oleg Platonov, Denis Kuznedelev, Michael Diskin, Artem Babenko, and Liudmila Prokhorenkova. 2023. A critical look at the evaluation of GNNs under heterophily: Are we really making progress?. In *The Eleventh International Conference on Learning Representations*.

[27] Taoyang Qin, Ke-Jia Chen, and Zheng Liu. 2025. Localized Heat Kernel for Graph Neural Networks. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*. Springer, 457–473.

[28] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. 2008. Collective classification in network data. *AI magazine* 29, 3 (2008), 93–93.

[29] Yunchong Song, Chenghu Zhou, Xinbing Wang, and Zhouhan Lin. 2023. Ordered GNN: Ordering Message Passing to Deal with Heterophily and Over-smoothing. In *ICLR*.

[30] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations*.

[31] Junfu Wang, Yuanfang Guo, Liang Yang, and Yunhong Wang. 2024. Heterophily-aware graph attention network. *Pattern Recognition* 156 (2024), 110738.

[32] Tao Wang, Di Jin, Rui Wang, Dongxiao He, and Yuxiao Huang. 2022. Powerful graph convolutional networks with adaptive propagation mechanism for homophily and heterophily. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 36. 4210–4218.

[33] Yu Wang and Tyler Derr. 2021. Tree decomposed graph neural network. In *Proceedings of the 30th ACM international conference on information & knowledge management*. 2040–2049.

[34] Jianshe Wu, Yaolin Liu, Yuqian Wang, Lingjie Zhang, and Jingyi Ding. 2025. HGphormer: Heterophilic Graph Transformer. *Knowledge-Based Systems* (2025), 114031.

[35] Yang Wu, Yu Wang, Liang Hu, and Juncheng Hu. 2024. DCGNN: Adaptive deep graph convolution for heterophily graphs. *Information Sciences* 666 (2024), 120427.

[36] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and Philip S Yu. 2020. A comprehensive survey on graph neural networks. *IEEE transactions on neural networks and learning systems* 32, 1 (2020), 4–24.

[37] Yujun Yan, Milad Hashemi, Kevin Swersky, Yaoqing Yang, and Danai Koutra. 2022. Two sides of the same coin: Heterophily and oversmoothing in graph convolutional neural networks. In *2022 IEEE International Conference on Data Mining (ICDM)*. IEEE, 1287–1292.

[38] Liangwei Yang, Zhiwei Liu, Yingtong Dou, Jing Ma, and Philip S Yu. 2021. Consisrec: Enhancing gnn for social recommendation via consistent neighbor aggregation. In *Proceedings of the 44th international ACM SIGIR conference on Research and development in information retrieval*. 2141–2145.

[39] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting semi-supervised learning with graph embeddings. In *International conference on machine learning*. PMLR, 40–48.

[40] Shichang Zhang, Jiani Zhang, Xiang Song, Soji Adeshina, Da Zheng, Christos Faloutsos, and Yizhou Sun. 2023. PaGE-Link: Path-based graph neural network explanation for heterogeneous link prediction. In *Proceedings of the ACM web conference 2023*. 3784–3793.

[41] Zhuonan Zheng, Yuanchen Bei, Sheng Zhou, Yao Ma, Ming Gu, Hongjia Xu, Chengyu Lai, Jiawei Chen, and Jiajun Bu. 2024. Revisiting the message passing in heterophilous graph neural networks. *arXiv preprint arXiv:2405.17768* (2024).

[42] Zhuonan Zheng, Sheng Zhou, Hongjia Xu, Ming Gu, Yilun Xu, Ao Li, Yuhong Li, Jingjun Gu, and Jiajun Bu. 2025. Heterophilous distribution propagation for Graph Neural Networks. *Neural Networks* 184 (2025), 107014.

[43] Jiong Zhu, Yujun Yan, Lingxiao Zhao, Mark Heimann, Leman Akoglu, and Danai Koutra. 2020. Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in neural information processing systems* 33 (2020), 7793–7804.


# A Proof and Analysis of Theorem 2

## A.1 Proof

**Theorem 2.** Consider a graph $G \sim CSBM(\mu_0, \mu_1, p, q)$, the amount of added conflicting nodes is $\alpha$ times that of each node's degree, and the accuracy of estimating the conflicting node's class type (the same class or different classes) is $e$. For any node $i$ in this

graph, the linear classifier defined by the decision boundary $P$ has a lower probability to misclassify $h_i$ of CNDGNN than $x_i$ when $deg(i) > \frac{1+\alpha}{[\frac{p-q}{p+q}+\alpha(2e-1)]^2}$.

*proof.* We first derive the new Gaussian distribution of node embeddings $h_i$ through our method. Compared with vanilla GNN, our method additionally aggregates $k = \alpha \cdot deg(i)$ conflicting nodes with estimation accuracy of $e$, and the aggregation weights are $\frac{1}{deg(i)+k}$. So the expected value of $h_i \in Class_0$ is calculated as follows.

$$\begin{aligned} E(h_i) &= \frac{pu_0 + qu_1}{p+q}\frac{\deg(i)}{\deg(i)+k} \\ &\quad + \frac{p[eu_0 + (1-e)u_1] - q[eu_1 + (1-e)u_0]}{p+q}\frac{k}{\deg(i)+k} \\ &= \frac{[\deg(i)p + kpe - kq(1-e)]u_0}{(p+q)(\deg(i)+k)} \\ &\quad + \frac{[\deg(i)q + kp(1-e) - kqe]u_1}{(p+q)(\deg(i)+k)} \end{aligned} \tag{18}$$

The features $h_i$ obtained from our method follow the Gaussian distribution:

$$h_i \sim N(E(h_i), \frac{1}{\sqrt{deg(i)+k}}I), \quad for\ i \in Class_0 \tag{19}$$

Consistent with theorem 1, we compare the probability that the linear classifier misclassifies a certain node based on the features before and after the GNN operation. The decision boundary is $P = \{w^\top x - w^\top(u_0+u_1)/2\}$, and $w = (u_0-u_1)/||u_0-u_1||_2$. We only prove for nodes from $Class_0$ since the case for nodes from $Class_1$ is symmetric and the proof is exactly the same. For a node $i \in Class_0$, we have the following.

$$P\big(x_i \text{ is misclassified}\big) = P\Big(w^\top x_i + b \le 0\Big), \quad \text{for } i \in \text{Class}_0 \tag{20}$$

$$P\big(h_i \text{ is misclassified}\big) = P\Big(w^\top h_i + b \le 0\Big), \quad \text{for } i \in \text{Class}_0 \tag{21}$$

where $w$ and $b = -w^\top(u_0+u_1)/2$ are the parameters of the decision boundary $P$. We have that

$$P\Big(w^\top h_i + b \le 0\Big) = P\Big(w^\top\sqrt{deg(i)+k}\,h_i + \sqrt{deg(i)+k}\,b \le 0\Big) \tag{22}$$

We denote the scaled version of $h_i$ as $h_i' = \sqrt{deg(i)+k}\,h_i$, and $h_i'$ allows

$$h_i \sim N(\sqrt{deg(i)+k}\,E(h_i), I), \quad for\ i \in Class_0 \tag{23}$$

Because of the scale, the decision boundary of $h_i'$ is correspondingly moved to $w^\top h_i' + \sqrt{\deg(i)+k}\,b = 0$. Since $x_i$ and $h_i'$ share the same variance, to compare the misclassification probabilities, we only need to compare the distance from their expected value to their corresponding decision boundary. The two distances are as follows.

$$dis_{x_i} = \frac{||u_0-u_1||_2}{2} \tag{24}$$

$$\begin{aligned} dis_{h_i'} &= \frac{\sqrt{\deg(i)+k}\,|\deg(i)(p-q) + kp(2e-1) + kq(2e-1)|}{(p+q)(\deg(i)+k)} \\ &\quad \cdot \frac{||u_0-u_1||_2}{2} \\ &= \frac{|\deg(i)(p-q) + k(2e-1)(p+q)|}{(p+q)\sqrt{deg(i)+k}} \cdot \frac{||u_0-u_1||_2}{2} \end{aligned} \tag{25}$$

The larger the distance is, the smaller the misclassification probability is. Hence, when $dis_{h_i'} > dis_{x_i}$, $h_i'$ has a lower probability to be misclassified than $x_i$.

$$\begin{aligned} \frac{|\deg(i)(p-q) + k(2e-1)(p+q)|}{(p+q)\sqrt{\deg(i)+k}} &> 1 \\ \frac{|\deg(i)\frac{p-q}{p+q} + k(2e-1)|}{\sqrt{\deg(i)+k}} &> 1 \\ \frac{|\deg(i)\frac{p-q}{p+q} + \deg(i)\alpha(2e-1)|}{\sqrt{\deg(i)+\deg(i)\alpha}} &> 1 \\ \sqrt{\deg(i)}\frac{\left|\frac{p-q}{p+q} + \alpha(2e-1)\right|}{\sqrt{1+\alpha}} &> 1 \\ \deg(i) &> \frac{1+\alpha}{\left[\frac{p-q}{p+q} + \alpha(2e-1)\right]^2} \end{aligned}$$

So we conclude that

$$P\big(h_i \text{ is misclassified}\big) < P\big(x_i \text{ is misclassified}\big) \tag{26}$$

$$\text{if } \deg(i) > \frac{1+\alpha}{\left[\frac{p-q}{p+q} + \alpha(2e-1)\right]^2} \tag{27}$$

which completes the proof.

## A.2 Analysis

Before analyzing Theorem 2, we will first discuss the parameter $\alpha$. $\alpha$ represents the proportion of added conflicting nodes to the node degree. It means each node additionally aggregates $k = \alpha \cdot deg(i)$ conflicting nodes. In our proof, the aggregation weights of conflicting nodes are fixed as $\frac{1}{deg(i)+k}$ for simplicity of computation. However, the aggregation weights of conflicting nodes in the conflicting embedding discrimination module are learned through the activation function $tanh$, ranging from -1 to 1, and the learned weights can actually be much larger than $\frac{1}{deg(i)+k}$. If the learned aggregation weight is 1 for a conflicting node, the effect of aggregating this node with the weight of 1 is equivalent to aggregating $deg(i)+k$ nodes with the weight of $\frac{1}{deg(i)+k}$, and the equivalent $\alpha$ for this node is $\frac{deg(i)+k}{deg(i)}$ in this case. So based on the analysis in this case, the actual parameter $\alpha$ can be seen as a relatively large positive number in Theorem 2.

For Theorem 1, as discussed in Section 4.3, $(p+q)^2/(p-q)^2$ tends towards positive infinity for the conflicting nodes ( $p = q$ in this case ), and the requirement $deg(i) > (p+q)^2/(p-q)^2$ is impossible to meet. For Theorem 2, when $p = q$, $\frac{1+\alpha}{\left[\frac{p-q}{p+q}+\alpha(2e-1)\right]^2} = \frac{1+\alpha}{[\alpha(2e-1)]^2}$. In this case, the requirement $deg(i) > \frac{1+\alpha}{[\alpha(2e-1)]^2}$ is easy to meet. The only exception is when $e = 0.5$, and it is hard to happen as $e$ can be optimized to avoid the condition in our method. Moreover, for the term $\frac{1+\alpha}{[\alpha(2e-1)]^2}$, its numerator is a linear function about $\alpha$

**Table 4: More details about datasets. |CND|, and |M| represent the node amount of conflicting node set CND and M, respectively.**

| Dataset | Roman-Empire | Amazon-Ratings | Chameleon-F | Squirrel-F | Flickr | Wikics | Pubmed | Photo |
|---|---|---|---|---|---|---|---|---|
| Homophily | 0.05 | 0.38 | 0.25 | 0.22 | 0.24 | 0.65 | 0.8 | 0.83 |
| Nodes | 22662 | 24492 | 890 | 2223 | 7575 | 11701 | 19717 | 7650 |
| Edges | 65854 | 186100 | 13584 | 65718 | 479476 | 431206 | 88651 | 238162 |
| Average Degree | 2.9 | 7.6 | 15.3 | 29.6 | 63.3 | 36.9 | 4.5 | 31.1 |
| Classes | 18 | 5 | 5 | 5 | 9 | 10 | 3 | 8 |
| Attributes | 300 | 300 | 2325 | 2089 | 12047 | 300 | 500 | 745 |
| \|CND\| | 18689 | 21261 | 288 | 725 | 263 | 2881 | 17472 | 1055 |
| \|M\| | 8052 | 10982 | 141 | 366 | 124 | 774 | 3342 | 207 |

and its denominator is a quadratic function about $\alpha$. According to the monotonicity of this term, the larger positive $\alpha$ contributes to smaller value of $\frac{1+\alpha}{[\alpha(2e-1)]^2}$, which makes the requirement $deg(i) > \frac{1+\alpha}{[\alpha(2e-1)]^2}$ easier to meet. In conclusion, our method performs better than GCN in the conflicting nodes.

### A.3 Designing Views

Based on previous analysis, we can conclude two key views when solving conflicting node problems. The first one is designing the module that can output a larger $\alpha$. A larger $\alpha$ makes the requirement $deg(i) > \frac{1+\alpha}{[\alpha(2e-1)]^2}$ easier to meet, and its benefit is analyzed in our case. The larger $\alpha$ is achieved mainly through the activation function $tanh$ in our method, and it deserves to be further explored, like designing other activation functions or the weight learning modules. The second one is improving the accuracy $e$ of estimating the conflicting node's class type. Although the value of $\frac{p-q}{p+q}$ ranges from -1 to 1, the high accuracy $e$ incorporated with large positive number $\alpha$ is able to make $\left[\frac{p-q}{p+q} + \alpha(2e-1)\right]$ ignore the condition of negative $\frac{p-q}{p+q}$ and constantly be a relatively large positive number, thus making the requirement $deg(i) > \frac{1+\alpha}{[\alpha(2e-1)]^2}$ easier to meet. To attain a higher value of $e$, we can explore more features to discriminate conflicting nodes, like the high-order neighbor distribution and original feature in our method. The known labels can also be well used. As the proportion of known labels is set as 48% in the usual semi-supervised node classification tasks, the accuracy $e$ can easily exceed 0.5 to reach a relatively high value if the known labels are taken advantage of, thus bringing more bonus to the improvement of accuracy $e$.

## B Dataset and Analysis

This section introduces more details about each dataset, especially the node amount of conflicting node set CND and M.

It can be seen from the results in Table 4 that most of the mainstream datasets contain high proportion of conflicting nodes. Flickr dataset contains relatively less conflicting nodes and attains less improvement from our modules. Pubmed owns only 3 node classes and 4.5 node neighbors on average, which are much fewer than other datasets. The statistics feature of Pubmed brings about fewer combination categories of neighbor distribution (19717 nodes with 3 labels squeeze between only dozens of neighbor distributions under the average degree), thus restricting the upper limit of neighbor distribution's discriminative power for Pubmed. So although Pubmed owns high proportion of conflicting nodes, it also attains less improvements from our modules.